GOODDEED

How to build the app, start to finish

A beginner-friendly build guide for Android and iOS, using React Native, Expo and Supabase

Make kindness the new social currency.

This guide turns the GOODDEED startup blueprint into a build plan: what to decide first, which tools to install, the database and security rules, the code for each MVP feature, testing, publishing to the app stores, and how to run the pilot. Code snippets are starting points. Library versions change, so check the current official documentation when a command or import behaves differently.

November 2026 · Planning estimates only, not legal advice

Contents

1. What you are building 3
2. Before you write code 4
3. Install your tools 4
4. Create the app 5
5. Create the Supabase project 6
6. Design the database 6
7. Lock it down with Row Level Security 9
8. Sign up and sign in 11
9. The home feed 12
10. Create a good deed, with photos 13
11. Appreciation and comments 14
12. Help requests and offers 14
13. Recipient confirmation without exposing the recipient 15
14. Reputation, badges and the leaderboard 17
15. Report, block and moderate 18
16. Privacy and safety build checklist 18
17. Test it properly 20
18. Publish to Google Play and the App Store 21
19. Run the pilot and measure what matters 21
20. After version 1 22
21. A 16-week plan 23
Appendix A. When something breaks 23
Appendix B. Glossary 24

PART A · PLAN

1. What you are building

GOODDEED is a mobile social platform where people earn recognition for real acts of kindness. Version 1 (the MVP) has 13 features. Everything else, such as kindness chains, daily challenges and sponsored campaigns, waits for version 2.

The guiding rule from the blueprint shapes every technical decision: **reward genuine kindness, not the performance of kindness.** In code, that means likes and followers never create reputation, recipients are never forced to appear publicly, and reputation is written only by the server.

The MVP and where each feature lives

MVP feature	Built with	Chapter
Registration and login	Supabase Auth + Expo screens	8
User profile	profiles table + profile screen	6, 8
Create good deed posts, upload images	posts, post_media, Storage	10
Home feed, like and comment	feed_posts view, reactions, comments	9, 11
Request help, offer help	help_requests, help_responses	12
Recipient confirmation	One-time code + Edge Function	13
Basic badges and recognition	Database triggers	14
Basic leaderboard	weekly_leaderboard view	14
Report and block users	reports, blocks	15
Admin moderation dashboard	moderator role + queue screen	15

The stack in one picture

Architecture

```
Mobile app (React Native + Expo, TypeScript)   <- Android and iOS from one codebase
       |
       |  supabase-js (HTTPS, signed-in user's token)
       v
Supabase
  Auth ............ sign up, sign in, sessions
  PostgreSQL ...... tables, views, triggers, Row Level Security (RLS)
  Storage ......... photos and videos
  Edge Functions .. confirm-deed, delete-account (server-only secrets)
```

The one idea to remember
The app on a phone can be modified by anyone who owns that phone. So the app is only a screen. The rules (who can see what, who can earn reputation) live in the database, where users cannot change them.

2. Before you write code

The blueprint is clear that a startup should confirm demand before spending months building. Do these four things first. They take two to three weeks and save much more than that.

2.1 Validate the idea

- Interview 15 to 20 potential users. Target: 20 interviews, 10 prototype testers, 5 organizations contacted.
- Ask: Have you helped a stranger? Would you share it? Would you confirm that someone helped you? Would you use an app to find nearby people who need help? What would make you trust or distrust the platform?
- Ask whether they would actually use it, not whether they like the idea.
- Research existing kindness, volunteering and social-impact apps. Do not claim there is no competition.

2.2 Write a short Product Requirements Document (PRD)

One document that lists every screen, every user role (member, recipient, moderator), every business rule and every database need. Use this skeleton and fill it in as you interview:

- Goal and non-goals for version 1
- User roles and what each can do
- The ten screens (Welcome, Sign up, Home, Explore, Create, Post details, Profile, Leaderboard, Notifications, Settings)
- Business rules (no points for likes, followers or money; verification raises trust; anonymous deeds allowed)
- Safety rules (no automatic exact location, recipient can remove a story)
- Success metrics (Chapter 19)

2.3 Brand and name

Finalize the name, logo, colors and typography. Check trademark and domain availability before committing. A domain costs roughly Rs. 800 to Rs. 1,500 a year.

2.4 Wireframes in Figma

Design the ten screens and make a clickable prototype. Test the full help-to-confirmation journey with at least one person before building it. Figma has a free tier.

PART B · BUILD

3. Install your tools

Tool	Why	Check it works
Node.js (LTS version)	Runs the JavaScript tooling	`node -v`
Git	Version control	`git --version`
VS Code	Code editor	Open it

Tool	Why	Check it works
GitHub account	Stores and backs up your code	Create a private repository named gooddeed
Expo Go (phone app)	Runs your app on a real phone while developing	Install from the Play Store or App Store
Android Studio (optional)	Android emulator on your computer	Needed only if you want an emulator
Xcode (Mac only, optional)	iOS simulator	Not required: cloud builds (Chapter 18) can produce iOS builds without a Mac

Learning path for a beginner, in order: JavaScript and TypeScript basics, React basics (components, state, effects), then React Native with Expo, then SQL basics. Official docs: docs.expo.dev, reactnative.dev, supabase.com/docs, typescriptlang.org/docs.

4. Create the app

Terminal

```
npx create-expo-app@latest gooddeed
cd gooddeed
npx expo install @react-native-async-storage/async-storage expo-image-picker \
  expo-image-manipulator
npm install @supabase/supabase-js react-native-url-polyfill
npx expo start
```

Scan the QR code with Expo Go. You should see the starter app on your phone. The default template uses TypeScript and file-based routing (Expo Router): every file inside the `app` folder becomes a screen.

Folder layout to aim for

Project structure

```
gooddeed/
  app/
    _layout.tsx           root layout, redirects to login when signed out
    (auth)/login.tsx      sign in and sign up
    (tabs)/index.tsx      Home feed
    (tabs)/explore.tsx    help requests and communities
    (tabs)/create.tsx     new deed or help request
    (tabs)/ranks.tsx      leaderboard
    (tabs)/profile.tsx    contribution profile and badges
    post/[id].tsx         post details, comments, verification
    settings.tsx          privacy, block list, delete account
  lib/
    supabase.ts           the one Supabase client
    session.tsx           who is signed in
  .env                    your Supabase URL and anon key (never commit)
```

Starter template note
The default template changes between Expo versions and may ship demo screens. Keep the idea of the layout above and adapt file names to what the template gives you.

5. Create the Supabase project

1. Create a free account at supabase.com and click New project.
2. Choose the region closest to your users (for India, a South Asia region).
3. Save the database password somewhere safe.
4. Open Project Settings, then API. Copy the Project URL and the anon (public) key.
5. Create a file named `.env` in the app folder and add the two values below.
6. Add `.env` to `.gitignore`.

.env

```
EXPO_PUBLIC_SUPABASE_URL=https://YOUR-PROJECT.supabase.co
EXPO_PUBLIC_SUPABASE_ANON_KEY=your-anon-key
```

Two keys, two very different rules
The **anon key** is meant to be inside the app. The **service_role key** bypasses all security rules. Never put it in the app, in GitHub, or in a screenshot. It is used only inside Edge Functions.

lib/supabase.ts

```
import 'react-native-url-polyfill/auto';
import AsyncStorage from '@react-native-async-storage/async-storage';
import { createClient } from '@supabase/supabase-js';

export const supabase = createClient(
  process.env.EXPO_PUBLIC_SUPABASE_URL!,
  process.env.EXPO_PUBLIC_SUPABASE_ANON_KEY!,
  {
    auth: {
      storage: AsyncStorage,
      autoRefreshToken: true,
      persistSession: true,
      detectSessionInUrl: false,
    },
  }
);
```

Restart Expo with `npx expo start -c` after editing .env. Variables must start with `EXPO_PUBLIC_` to reach the app.

6. Design the database

Open the Supabase SQL Editor and run these scripts in order. Supabase already stores accounts in a private `auth.users` table, so the blueprint's "users" table maps to that, and `profiles` holds public information.

SQL 1: core tables

```
create table public.profiles (
  id uuid primary key references auth.users(id) on delete cascade,
  username text unique not null check (char_length(username) between 3 and 30),
  display_name text,
  avatar_url text,
  city text,
  bio text,
  role text not null default 'user' check (role in ('user','moderator')),
  hide_ranking boolean not null default false,
  created_at timestamptz not null default now()
);

create table public.posts (
```

```
  id uuid primary key default gen_random_uuid(),
  author_id uuid not null references public.profiles(id) on delete cascade,
  category text not null check (category in ('food','education','transport',
    'medical','elderly','emergency','skills','community','other')),
  body text not null check (char_length(body) between 10 and 2000),
  is_anonymous boolean not null default false,
  visibility text not null default 'public'
    check (visibility in ('public','followers','private')),
  approx_area text,          -- a neighbourhood name, never exact coordinates
  status text not null default 'self_reported'
    check (status in ('self_reported','recipient_confirmed','community_reviewed')),
  created_at timestamptz not null default now()
);

create table public.post_media (
  id uuid primary key default gen_random_uuid(),
  post_id uuid not null references public.posts(id) on delete cascade,
  path text not null,
  kind text not null check (kind in ('image','video'))
);

create table public.help_requests (
  id uuid primary key default gen_random_uuid(),
  requester_id uuid not null references public.profiles(id) on delete cascade,
  category text not null,
  title text not null check (char_length(title) between 5 and 120),
  details text,
  approx_area text,
  status text not null default 'open'
    check (status in ('open','matched','completed','cancelled')),
  created_at timestamptz not null default now()
);

create table public.help_responses (
  id uuid primary key default gen_random_uuid(),
  request_id uuid not null references public.help_requests(id) on delete cascade,
  helper_id uuid not null references public.profiles(id) on delete cascade,
  message text,
  status text not null default 'offered'
    check (status in ('offered','accepted','declined')),
  created_at timestamptz not null default now(),
  unique (request_id, helper_id)
);
```

SQL 2: social, trust and safety tables

```
create table public.verifications (
  id uuid primary key default gen_random_uuid(),
  post_id uuid not null unique references public.posts(id) on delete cascade,
  code_hash text not null,           -- we store a hash, never the code itself
  expires_at timestamptz not null,
  confirmed_at timestamptz
);

create table public.reactions (
  post_id uuid references public.posts(id) on delete cascade,
  user_id uuid references public.profiles(id) on delete cascade,
  created_at timestamptz not null default now(),
  primary key (post_id, user_id)
);

create table public.comments (
  id uuid primary key default gen_random_uuid(),
  post_id uuid not null references public.posts(id) on delete cascade,
  user_id uuid not null references public.profiles(id) on delete cascade,
  body text not null check (char_length(body) between 1 and 500),
  created_at timestamptz not null default now()
);

create table public.follows (
  follower_id uuid references public.profiles(id) on delete cascade,
  following_id uuid references public.profiles(id) on delete cascade,
  primary key (follower_id, following_id)
);
```

```
create table public.blocks (
  blocker_id uuid references public.profiles(id) on delete cascade,
  blocked_id uuid references public.profiles(id) on delete cascade,
  primary key (blocker_id, blocked_id)
);

create table public.badges (
  code text primary key, name text not null, description text not null
);
create table public.user_badges (
  user_id uuid references public.profiles(id) on delete cascade,
  badge_code text references public.badges(code),
  earned_at timestamptz not null default now(),
  primary key (user_id, badge_code)
);

create table public.reputation_events (
  id bigint generated always as identity primary key,
  user_id uuid not null references public.profiles(id) on delete cascade,
  kind text not null check (kind in ('deed_posted','deed_confirmed','report_upheld')),
  post_id uuid references public.posts(id) on delete set null,
  created_at timestamptz not null default now()
);

create table public.reports (
  id uuid primary key default gen_random_uuid(),
  reporter_id uuid not null references public.profiles(id) on delete cascade,
  target_type text not null check (target_type in ('post','comment','user','help_request')),
  target_id uuid not null,
  reason text not null,
  details text,
  status text not null default 'open'
    check (status in ('open','reviewing','actioned','dismissed')),
  created_at timestamptz not null default now()
);

create table public.notifications (
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null references public.profiles(id) on delete cascade,
  kind text not null, post_id uuid references public.posts(id) on delete cascade,
  is_read boolean not null default false,
  created_at timestamptz not null default now()
);

insert into public.badges values
  ('first_kindness','First Kindness','First completed good deed'),
  ('consistency_star','Consistency Star','Regular positive actions'),
  ('community_hero','Community Hero','Community contributions'),
  ('trusted_helper','Trusted Helper','Consistent verified activity');
```

SQL 3: create a profile automatically when someone signs up

```
create function public.handle_new_user()
returns trigger language plpgsql security definer set search_path = public as $$
begin
  insert into public.profiles (id, username, display_name)
  values (new.id,
          coalesce(new.raw_user_meta_data->>'username',
                   'user_' || substr(new.id::text, 1, 8)),
          new.raw_user_meta_data->>'display_name');
  return new;
end $$;

create trigger on_auth_user_created
  after insert on auth.users
  for each row execute function public.handle_new_user();
```

7. Lock it down with Row Level Security

Row Level Security (RLS) is a set of rules inside the database that decides which rows each signed-in user may read, add, change or delete. With RLS on, a table with no matching rule is closed. Turn it on for every table before you write any app code, so you never ship something open by accident.

SQL 4: switch RLS on everywhere

```
alter table public.profiles          enable row level security;
alter table public.posts             enable row level security;
alter table public.post_media        enable row level security;
alter table public.help_requests     enable row level security;
alter table public.help_responses    enable row level security;
alter table public.verifications     enable row level security;  -- no policies: closed
alter table public.reactions         enable row level security;
alter table public.comments          enable row level security;
alter table public.follows           enable row level security;
alter table public.blocks            enable row level security;
alter table public.badges            enable row level security;
alter table public.user_badges       enable row level security;
alter table public.reputation_events enable row level security;
alter table public.reports           enable row level security;
alter table public.notifications     enable row level security;
```

7.1 Profiles and posts

SQL 5: profile and post rules

```
create policy "profiles readable" on public.profiles
  for select to authenticated using (true);
create policy "profiles update own" on public.profiles
  for update to authenticated using (id = auth.uid()) with check (id = auth.uid());
-- people may edit these columns only (so nobody can make themselves a moderator)
revoke update on public.profiles from authenticated;
grant update (username, display_name, avatar_url, city, bio, hide_ranking)
  on public.profiles to authenticated;

create policy "posts read own" on public.posts
  for select to authenticated using (author_id = auth.uid());
create policy "posts insert own" on public.posts
  for insert to authenticated
  with check (author_id = auth.uid() and status = 'self_reported');
create policy "posts update own" on public.posts
  for update to authenticated
  using (author_id = auth.uid()) with check (author_id = auth.uid());
create policy "posts delete own" on public.posts
  for delete to authenticated using (author_id = auth.uid());
-- the author can never edit "status": only the server confirms deeds
revoke update on public.posts from authenticated;
grant update (body, visibility, is_anonymous, approx_area)
  on public.posts to authenticated;
```

7.2 The feed view: one safe door for reading posts

Notice that the policy above lets people read only their *own* rows in `posts`. Everyone else reads through a view that applies the visibility, block and anonymity rules in one place. For anonymous posts the view returns no author, so the author's identity never reaches other people's phones.

SQL 6: feed_posts view

```
create view public.feed_posts as
select p.id,
       case when p.is_anonymous then null else p.author_id end as author_id,
       case when p.is_anonymous then null else pr.username end as username,
       p.category, p.body, p.approx_area, p.status, p.created_at
from public.posts p
join public.profiles pr on pr.id = p.author_id
where (p.visibility = 'public'
```

```
      or p.author_id = auth.uid()
      or (p.visibility = 'followers' and exists (
            select 1 from public.follows f
            where f.follower_id = auth.uid() and f.following_id = p.author_id)))
  and not exists (
       select 1 from public.blocks b
       where (b.blocker_id = auth.uid() and b.blocked_id = p.author_id)
          or (b.blocker_id = p.author_id and b.blocked_id = auth.uid())));

revoke all on public.feed_posts from anon;
grant select on public.feed_posts to authenticated;
```

About the "security definer view" warning
A normal view runs with its owner's permissions and so skips RLS on the tables underneath. Here that is on purpose: the view does its own filtering. Supabase's advisor may flag it. Keep the filter conditions in the view exactly as written and test them with two accounts (Chapter 17).

7.3 Everything else

SQL 7: remaining rules

```
create function public.is_moderator() returns boolean
language sql security definer stable set search_path = public as $$
  select exists (select 1 from public.profiles
                 where id = auth.uid() and role = 'moderator');
$$;

-- media
create policy "media read" on public.post_media for select to authenticated
  using (exists (select 1 from public.feed_posts f where f.id = post_media.post_id)
         or exists (select 1 from public.posts p
                    where p.id = post_media.post_id and p.author_id = auth.uid()));
create policy "media insert own" on public.post_media for insert to authenticated
  with check (exists (select 1 from public.posts p
              where p.id = post_media.post_id and p.author_id = auth.uid()));

-- reactions and comments: only on posts you are allowed to see
create policy "reactions read" on public.reactions for select to authenticated
  using (exists (select 1 from public.feed_posts f where f.id = reactions.post_id));
create policy "reactions add" on public.reactions for insert to authenticated
  with check (user_id = auth.uid()
    and exists (select 1 from public.feed_posts f where f.id = reactions.post_id));
create policy "reactions remove" on public.reactions for delete to authenticated
  using (user_id = auth.uid());

create policy "comments read" on public.comments for select to authenticated
  using (exists (select 1 from public.feed_posts f where f.id = comments.post_id));
create policy "comments add" on public.comments for insert to authenticated
  with check (user_id = auth.uid()
    and exists (select 1 from public.feed_posts f where f.id = comments.post_id));
create policy "comments delete own" on public.comments for delete to authenticated
  using (user_id = auth.uid());

-- follows and blocks are private to the person who made them
create policy "follows own" on public.follows for all to authenticated
  using (follower_id = auth.uid()) with check (follower_id = auth.uid());
create policy "blocks own" on public.blocks for all to authenticated
  using (blocker_id = auth.uid()) with check (blocker_id = auth.uid());

-- badges are public; badges and reputation are written only by triggers
create policy "badges read" on public.badges for select to authenticated using (true);
create policy "user_badges read" on public.user_badges for select to authenticated using (true);
create policy "reputation read own" on public.reputation_events
  for select to authenticated using (user_id = auth.uid());

-- reports: anyone can file one, only moderators read the queue
create policy "reports file" on public.reports for insert to authenticated
  with check (reporter_id = auth.uid());
create policy "reports read" on public.reports for select to authenticated
  using (reporter_id = auth.uid() or public.is_moderator());
create policy "reports moderate" on public.reports for update to authenticated
```

```
  using (public.is_moderator()) with check (public.is_moderator());

-- notifications
create policy "notif read own" on public.notifications
  for select to authenticated using (user_id = auth.uid());
create policy "notif mark read" on public.notifications
  for update to authenticated using (user_id = auth.uid());
```

8. Sign up and sign in

lib/session.tsx

```
import { createContext, useContext, useEffect, useState } from 'react';
import type { Session } from '@supabase/supabase-js';
import { supabase } from './supabase';

type Ctx = { session: Session | null; ready: boolean };
const SessionContext = createContext<Ctx>({ session: null, ready: false });
export const useSession = () => useContext(SessionContext);

export function SessionProvider({ children }: { children: React.ReactNode }) {
  const [session, setSession] = useState<Session | null>(null);
  const [ready, setReady] = useState(false);

  useEffect(() => {
    supabase.auth.getSession().then(({ data }) => {
      setSession(data.session);
      setReady(true);
    });
    const { data: sub } = supabase.auth.onAuthStateChange((_event, s) => {
      setSession(s);
    });
    return () => sub.subscription.unsubscribe();
  }, []);

  return (
    <SessionContext.Provider value={{ session, ready }}>
      {children}
    </SessionContext.Provider>
  );
}
```

app/_layout.tsx

```
import { Slot, useRouter, useSegments } from 'expo-router';
import { useEffect } from 'react';
import { SessionProvider, useSession } from '@/lib/session';

function Gate() {
  const { session, ready } = useSession();
  const segments = useSegments();
  const router = useRouter();

  useEffect(() => {
    if (!ready) return;
    const inAuth = segments[0] === '(auth)';
    if (!session && !inAuth) router.replace('/(auth)/login');
    if (session && inAuth) router.replace('/(tabs)');
  }, [session, ready, segments]);

  return <Slot />;
}

export default function Root() {
  return (
    <SessionProvider>
      <Gate />
    </SessionProvider>
  );
}
```

app/(auth)/login.tsx (the two calls that matter)

```
async function signUp() {
  const { error } = await supabase.auth.signUp({
    email,
    password,
    options: { data: { username, display_name: username } },
  });
  if (error) Alert.alert('Could not sign up', error.message);
}

async function signIn() {
  const { error } = await supabase.auth.signInWithPassword({ email, password });
  if (error) Alert.alert('Could not sign in', error.message);
}
```

Build a simple screen with a username field, an email field, a password field and two buttons. In the Supabase dashboard (Authentication) email confirmation is on by default. Leave it on for production. While testing, you may turn it off so sign-up creates a session immediately. Phone and social login can be added later in the same Authentication settings.

9. The home feed

app/(tabs)/index.tsx

```
import { useCallback, useEffect, useState } from 'react';
import { FlatList, RefreshControl, Text, View } from 'react-native';
import { supabase } from '@/lib/supabase';

type Post = {
  id: string; username: string | null; category: string;
  body: string; status: string; created_at: string;
};

const STATUS: Record<string, string> = {
  self_reported: 'Self-reported',
  recipient_confirmed: 'Recipient confirmed',
  community_reviewed: 'Community reviewed',
};

export default function Home() {
  const [posts, setPosts] = useState<Post[]>([]);
  const [loading, setLoading] = useState(false);

  const load = useCallback(async () => {
    setLoading(true);
    const { data, error } = await supabase
      .from('feed_posts').select('*')
      .order('created_at', { ascending: false }).limit(20);
    if (!error && data) setPosts(data as Post[]);
    setLoading(false);
  }, []);

  useEffect(() => { load(); }, [load]);

  return (
    <FlatList
      data={posts}
      keyExtractor={(p) => p.id}
      refreshControl={<RefreshControl refreshing={loading} onRefresh={load} />}
      renderItem={({ item }) => (
        <View style={{ padding: 16, borderBottomWidth: 1, borderColor: '#D3E0D8' }}>
          <Text style={{ fontWeight: '700' }}>{item.username ?? 'Anonymous'}</Text>
          <Text>{item.body}</Text>
          <Text style={{ color: '#11694B' }}>{STATUS[item.status]}</Text>
        </View>
      )}
    />
  );
}
```

Two details matter for trust. First, show the verification label exactly as stored and never claim more than the evidence checked. Second, show only the approximate area. Page the list with `.range(from, to)` once you have more than a few dozen posts.

10. Create a good deed, with photos

10.1 Storage bucket and rules

SQL 8: private media bucket

```
insert into storage.buckets (id, name, public) values ('post-media','post-media', false);

-- people may upload only into a folder named after their own user id
create policy "upload to own folder" on storage.objects
  for insert to authenticated
  with check (bucket_id = 'post-media'
              and (storage.foldername(name))[1] = auth.uid()::text);

-- people may view files that belong to posts they are allowed to see
create policy "view visible media" on storage.objects
  for select to authenticated
  using (bucket_id = 'post-media' and exists (
    select 1 from public.post_media m
    where m.path = storage.objects.name
      and (exists (select 1 from public.feed_posts f where f.id = m.post_id)
           or exists (select 1 from public.posts p
                      where p.id = m.post_id and p.author_id = auth.uid()))));
```

10.2 Pick, shrink, upload, publish

lib/deeds.ts

```
import * as ImagePicker from 'expo-image-picker';
import * as ImageManipulator from 'expo-image-manipulator';
import { supabase } from './supabase';

export async function pickImage(): Promise<string | null> {
  const res = await ImagePicker.launchImageLibraryAsync({
    mediaTypes: ['images'], quality: 1, exif: false,
  });
  if (res.canceled) return null;
  // Re-encoding also drops hidden metadata such as GPS tags
  const out = await ImageManipulator.manipulateAsync(
    res.assets[0].uri,
    [{ resize: { width: 1080 } }],
    { compress: 0.7, format: ImageManipulator.SaveFormat.JPEG }
  );
  return out.uri;
}

async function uploadImage(uri: string, userId: string) {
  const path = `${userId}/${Date.now()}.jpg`;
  const bytes = await (await fetch(uri)).arrayBuffer();
  const { error } = await supabase.storage
    .from('post-media').upload(path, bytes, { contentType: 'image/jpeg' });
  if (error) throw error;
  return path;
}

export async function publishDeed(userId: string, form: {
  category: string; body: string; anonymous: boolean;
  visibility: 'public' | 'followers' | 'private';
  area?: string; imageUri?: string | null;
}) {
  const { data: post, error } = await supabase.from('posts').insert({
    author_id: userId, category: form.category, body: form.body,
    is_anonymous: form.anonymous, visibility: form.visibility,
    approx_area: form.area || null,
```

```
  }).select('id').single();
  if (error) throw error;

  if (form.imageUri) {
    const path = await uploadImage(form.imageUri, userId);
    await supabase.from('post_media').insert({ post_id: post.id, path, kind: 'image' });
  }
  return post.id as string;
}
```

To show an image, ask for a temporary link: `supabase.storage.from('post-media').createSignedUrl(path, 3600)`.

10.3 The consent-first form

Build the Create screen from the blueprint fields: category, description, optional media, optional approximate location, verification and visibility. Add these protections in the interface itself:

- A visible switch: **Post anonymously**.
- A reminder above the photo button: do not show a recipient's face or identifying details without their permission.
- Location is a text field for a neighbourhood. Never read the phone's GPS for the post.
- A short note that the recipient may ask for the story to be made private or removed.

11. Appreciation and comments

Likes are friendly engagement only. They must never feed the reputation system.

Appreciate and comment

```
export async function toggleLike(postId: string, userId: string, liked: boolean) {
  if (liked) {
    await supabase.from('reactions').delete()
      .eq('post_id', postId).eq('user_id', userId);
  } else {
    await supabase.from('reactions').insert({ post_id: postId, user_id: userId });
  }
}

export async function addComment(postId: string, userId: string, body: string) {
  const { error } = await supabase.from('comments')
    .insert({ post_id: postId, user_id: userId, body });
  if (error) throw error;
}
```

For live updates, Supabase Realtime can push new comments to an open screen. Add it after the basics work.

12. Help requests and offers

This is what makes GOODDEED more useful than a normal social network. The workflow is: a person submits a request, helpers respond, the requester chooses, help happens, the request is marked completed, and both sides can optionally confirm.

SQL 9: rules for requests and offers

```
create policy "requests read" on public.help_requests for select to authenticated
  using (status = 'open' or requester_id = auth.uid());
create policy "requests create" on public.help_requests for insert to authenticated
  with check (requester_id = auth.uid() and status = 'open');
```

```
create policy "requests update own" on public.help_requests for update to authenticated
  using (requester_id = auth.uid()) with check (requester_id = auth.uid());
revoke update on public.help_requests from authenticated;
grant update (title, details, status) on public.help_requests to authenticated;

create policy "responses read" on public.help_responses for select to authenticated
  using (helper_id = auth.uid() or exists (
    select 1 from public.help_requests r
    where r.id = help_responses.request_id and r.requester_id = auth.uid()));
create policy "responses offer" on public.help_responses for insert to authenticated
  with check (helper_id = auth.uid() and exists (
    select 1 from public.help_requests r
    where r.id = help_responses.request_id and r.status = 'open'
      and r.requester_id <> auth.uid()));
-- only the requester decides whether to accept
create policy "responses decide" on public.help_responses for update to authenticated
  using (exists (select 1 from public.help_requests r
    where r.id = help_responses.request_id and r.requester_id = auth.uid()));
revoke update on public.help_responses from authenticated;
grant update (status) on public.help_responses to authenticated;
```

Request, offer, accept, complete

```
await supabase.from('help_requests').insert({
  requester_id: userId, category: 'elderly',
  title: 'Help carrying groceries up two floors', approx_area: 'Kondapur',
});

await supabase.from('help_responses')
  .insert({ request_id, helper_id: userId, message: 'I can come at 6 pm' });

// requester accepts one helper, then marks the request matched
await supabase.from('help_responses').update({ status: 'accepted' }).eq('id', responseId);
await supabase.from('help_requests').update({ status: 'matched' }).eq('id', request_id);

// later
await supabase.from('help_requests').update({ status: 'completed' }).eq('id', request_id);
```

Safety rules for this module
Medical emergencies must show a clear message to contact emergency services instead of posting a request. Do not show phone numbers or exact addresses in the request. For V1, keep contact inside the app and add safety guidance for meeting a stranger (public place, tell someone, trust your instincts). Add a visible Report button on every request.

13. Recipient confirmation without exposing the recipient

The recipient may not have an account, and must never be forced to create one or appear publicly. So the helper generates a one-time code, the helper shares it privately (message, spoken, or on a card), and the recipient enters it on a tiny web page. The server stores only a hash of the code, so even a database leak does not reveal live codes.

13.1 Database functions

SQL 10: create and confirm

```
create function public.create_verification(p_post uuid)
returns text language plpgsql security definer
set search_path = public, extensions as $$
declare v_code text;
begin
  if not exists (select 1 from posts where id = p_post and author_id = auth.uid()) then
    raise exception 'not your post';
  end if;
```

```
  if exists (select 1 from verifications where post_id = p_post
             and confirmed_at is not null) then
    raise exception 'already confirmed';
  end if;
  v_code := upper(substr(encode(gen_random_bytes(6), 'hex'), 1, 10));
  insert into verifications (post_id, code_hash, expires_at)
  values (p_post, encode(digest(v_code, 'sha256'), 'hex'), now() + interval '7 days')
  on conflict (post_id) do update
    set code_hash = excluded.code_hash, expires_at = excluded.expires_at;
  return v_code;
end $$;

create function public.confirm_deed(p_code text) returns boolean
language plpgsql security definer
set search_path = public, extensions as $$
declare v_post uuid;
begin
  update verifications set confirmed_at = now()
  where code_hash = encode(digest(upper(trim(p_code)), 'sha256'), 'hex')
    and confirmed_at is null and expires_at > now()
  returning post_id into v_post;
  if v_post is null then return false; end if;
  update posts set status = 'recipient_confirmed'
  where id = v_post and status = 'self_reported';
  return true;
end $$;

-- functions are open to everyone by default, so close them deliberately
revoke execute on function public.create_verification(uuid) from public, anon;
grant  execute on function public.create_verification(uuid) to authenticated;
revoke execute on function public.confirm_deed(text) from public, anon, authenticated;
```

13.2 Edge Function the recipient's page calls

supabase/functions/confirm-deed/index.ts

```
import { createClient } from 'npm:@supabase/supabase-js@2';

const cors = {
  'Access-Control-Allow-Origin': '*',
  'Access-Control-Allow-Headers': 'content-type',
};

Deno.serve(async (req) => {
  if (req.method === 'OPTIONS') return new Response('ok', { headers: cors });

  const { code } = await req.json();
  const admin = createClient(
    Deno.env.get('SUPABASE_URL')!,
    Deno.env.get('SUPABASE_SERVICE_ROLE_KEY')!
  );
  const { data, error } = await admin.rpc('confirm_deed', { p_code: String(code) });

  return new Response(JSON.stringify({ confirmed: data === true }), {
    status: error ? 400 : 200,
    headers: { ...cors, 'Content-Type': 'application/json' },
  });
});
```

Deploy

```
npx supabase login
npx supabase functions deploy confirm-deed --no-verify-jwt --project-ref YOUR_REF
```

- Build a one-field static web page (on any static host) that posts the code to this function and shows "Thank you, your confirmation was recorded".
- The function is public, so add attempt limiting before the pilot (for example, count failed attempts per IP and slow them down) to stop code guessing.

- In the app, the helper taps **Ask recipient to confirm**, which calls `supabase.rpc('create_verification', { p_post })` and shows the code to share.
- Confirmation is optional for the recipient and never a condition for receiving help.

14. Reputation, badges and the leaderboard

Reputation is computed by database triggers from trusted events. The app can never send a score. The blueprint asks for three separate measures (Good Deeds, Impact, Trust) rather than one number. Version 1 records the first two as events and shows Trust as the verification labels and badges.

SQL 11: events and badges

```
create function public.log_deed_events()
returns trigger language plpgsql security definer set search_path = public as $$
begin
  if tg_op = 'INSERT' then
    insert into reputation_events (user_id, kind, post_id)
    values (new.author_id, 'deed_posted', new.id);
    if (select count(*) from posts where author_id = new.author_id) = 1 then
      insert into user_badges (user_id, badge_code)
      values (new.author_id, 'first_kindness') on conflict do nothing;
    end if;
  elsif new.status = 'recipient_confirmed' and old.status = 'self_reported' then
    insert into reputation_events (user_id, kind, post_id)
    values (new.author_id, 'deed_confirmed', new.id);
    if (select count(*) from reputation_events
        where user_id = new.author_id and kind = 'deed_confirmed') >= 5 then
      insert into user_badges (user_id, badge_code)
      values (new.author_id, 'trusted_helper') on conflict do nothing;
    end if;
  end if;
  return new;
end $$;

create trigger deed_events
  after insert or update of status on public.posts
  for each row execute function public.log_deed_events();
```

SQL 12: profile numbers and weekly leaderboard

```
create view public.contribution_profile as
select pr.id as user_id,
       count(*) filter (where e.kind = 'deed_posted')    as good_deeds,
       count(*) filter (where e.kind = 'deed_confirmed') as confirmed
from public.profiles pr
left join public.reputation_events e on e.user_id = pr.id
group by pr.id;

create view public.weekly_leaderboard as
select e.user_id, pr.username, count(*) as confirmed
from public.reputation_events e
join public.profiles pr on pr.id = e.user_id
where e.kind = 'deed_confirmed'
  and e.created_at > now() - interval '7 days'
  and not pr.hide_ranking
group by e.user_id, pr.username
order by confirmed desc
limit 50;

grant select on public.contribution_profile, public.weekly_leaderboard to authenticated;
```

Reading them in the app

```
const { data: board } = await supabase.from('weekly_leaderboard').select('*');
const { data: me } = await supabase.from('contribution_profile')
  .select('*').eq('user_id', userId).single();
const { data: badges } = await supabase.from('user_badges')
  .select('badge_code, earned_at, badges(name, description)').eq('user_id', userId);
```

- The board ranks confirmed contributions only. It ignores likes, followers and money.
- Users can hide their ranking (the `hide_ranking` switch in Settings).
- Anti-fraud: cap how many deeds per day can create events (for example, extra deeds in one day earn no event) and penalize deeds that moderators mark as staged by adding a `report_upheld` event.
- Consistency Star and Community Hero need scheduled checks. Add them after launch with a daily scheduled database job.
- Month, city and community boards are the same query with a different date range or a city filter.

15. Report, block and moderate

The blueprint calls this essential and not optional. App stores also expect apps with user-generated content to offer reporting, blocking and a way to contact the developer.

In the app

```
// report a staged, misleading or exploitative deed without confronting anyone
await supabase.from('reports').insert({
  reporter_id: userId, target_type: 'post', target_id: postId,
  reason: 'staged_or_misleading', details,
});

// block a user (their posts and yours disappear from each other's feeds)
await supabase.from('blocks').insert({ blocker_id: userId, blocked_id: otherId });

// moderator queue (returns rows only for people with role = 'moderator')
const { data: queue } = await supabase.from('reports')
  .select('*').eq('status', 'open').order('created_at');
```

Making someone a moderator

In the SQL Editor (never from the app), run: `update profiles set role = 'moderator' where username = 'your_username';`

A moderation process that works

- A Reports screen visible only to moderators, with the post, the reporter's reason and three actions: Dismiss, Remove post, Remove and warn.
- Reasons offered to reporters: staged or misleading, exploits the recipient, harassment, spam or scam, unsafe, other.
- Reported abuse always gets a human decision. Write down a target response time and keep a log.
- A way to appeal: an Appeal button on removal notices that creates a new report row for another moderator.
- Start with Supabase Studio (the table viewer) as your admin tool, then build the in-app screen once you have real reports.

16. Privacy and safety build checklist

Blueprint requirement	How you build it
Never publish exact location	Store a neighbourhood text field only. No latitude or longitude columns. Re-encode photos to drop GPS metadata.

Blueprint requirement	How you build it
Never expose recipient information	Recipients have no profile. Confirmation uses a hashed one-time code. Anonymous posts hide the author in the feed view.
Blocking and reporting	blocks and reports tables, buttons on every post, request and comment.
Human moderation tools	moderator role, report queue, action log.
Safeguards for minors	Set a minimum age at sign-up, ask the age with a neutral prompt, avoid collecting extra data, and take legal advice on the rules for children's data.
Protect verification data	verifications table has RLS on and no policies. Only database functions touch it.
Account and data deletion	Settings screen button that calls a delete-account Edge Function (below).
Spam and scam detection	Daily limits per user, new-account limits on requests, and a report reason for scams. Add automated checks in V2.
Appeals	Appeal button on removal notices.

supabase/functions/delete-account/index.ts

```
import { createClient } from 'npm:@supabase/supabase-js@2';

Deno.serve(async (req) => {
  const url = Deno.env.get('SUPABASE_URL')!;
  const asUser = createClient(url, Deno.env.get('SUPABASE_ANON_KEY')!, {
    global: { headers: { Authorization: req.headers.get('Authorization')! } },
  });
  const { data: { user } } = await asUser.auth.getUser();
  if (!user) return new Response('Unauthorized', { status: 401 });

  const admin = createClient(url, Deno.env.get('SUPABASE_SERVICE_ROLE_KEY')!);
  await admin.auth.admin.deleteUser(user.id);   // cascades to profile, posts, etc.
  return new Response('ok');
});
```

Call it from Settings with `supabase.functions.invoke('delete-account')` after a confirmation screen. Also delete the person's files in the post-media bucket (a folder named with their user id) as part of the same flow.

India-specific legal review before public launch

- Digital Personal Data Protection Act and applicable rules
- Information Technology Act and intermediary obligations, where applicable
- Consumer protection requirements
- Content moderation and grievance redressal processes
- Children's privacy and age safeguards
- Fundraising, payment and charitable solicitation rules (only if you add donations later)
- Publish a privacy policy and terms of use, and give a contact for complaints

Take professional legal advice and get quotes. Do not collect identity documents or sensitive recipient information unless there is a clear legal and operational need. Version 1 has no money wallet.

17. Test it properly

17.1 Test the security rules with two accounts

Create two test users, A and B, on two phones (or one phone plus Expo Go in a browser). Go through every row below. Each of these should fail or return nothing.

Test	Expected result
B reads A's private post	Not visible in B's feed
B reads A's followers-only post without following	Not visible
A posts anonymously, B inspects the feed data	author_id and username are empty
A tries to update own post status to recipient_confirmed	Rejected
A inserts a post with status community_reviewed	Rejected
A changes own role to moderator	Rejected
A inserts a reputation event or badge directly	Rejected
B accepts a response to A's request	Rejected
After A blocks B, B opens the feed	A's posts are gone
Anyone calls confirm_deed from the app	Permission denied
A uploads a file into B's storage folder	Rejected
Confirm a deed with a wrong, expired, or already used code	Returns confirmed: false

17.2 Test the whole journey

1. A creates an account and publishes a deed with a photo.
2. A asks the recipient to confirm and shares the code.
3. The recipient confirms on the web page.
4. A's post shows Recipient confirmed, A gets a reputation event, the leaderboard moves.
5. B reports a different post, a moderator removes it, and the author can appeal.
6. A deletes the account and the data disappears.

17.3 Test on real devices

- Test on at least one low-cost Android phone and one iPhone.
- Try slow networks and airplane mode: errors should read clearly, not crash.
- Check small screens, large text settings, and dark mode.
- Use a free test channel: Google Play internal testing and Apple TestFlight.

PART C · SHIP

18. Publish to Google Play and the App Store

18.1 Cloud builds with EAS

Terminal

```
npm install -g eas-cli
eas login
eas build:configure
eas build --platform android --profile preview     # installable test build
eas build --platform ios --profile preview         # needs an Apple Developer account
```

EAS Build makes the Android and iOS files in the cloud, so you do not need a Mac for iOS builds. Set the app name and unique identifiers (for example `com.yourcompany.gooddeed`) in app.json before the first build, because they are hard to change later. Add your Supabase URL and anon key as EAS environment variables rather than committing them.

18.2 Store accounts and listings

- Google Play Console: a one-time registration fee. Apple Developer Program: an annual fee. Check both current prices before budgeting.
- Prepare: app icon, 4 to 8 screenshots per device type, a short and long description, a privacy policy web address, a support contact, and an age rating.
- Complete the data safety (Google) and privacy details (Apple) forms honestly: list photos, account data and any analytics.
- Make sure these are in the app before you submit: report, block, account deletion, privacy policy link, and moderation contact.
- Reviews can take days and may ask for changes. Submit early, and keep a demo account ready for the reviewers.

18.3 Updates

Small fixes to the app's JavaScript can be sent without a new store review using `eas update`. Changes to native settings, permissions or new native libraries need a new build.

1. Fix a bug, then run eas update to the preview channel and test.
2. Promote to production when it works.
3. Database changes: write them as SQL files and keep them in your repository so you can repeat them.

19. Run the pilot and measure what matters

Start small, as in the blueprint: private prototype, then a closed community of 50 to 100 people, then a local pilot of 300 to 500 users through colleges, community groups and NGOs, with Hyderabad as the pilot city. Create real stories, not just advertising.

The main number is **verified positive outcomes per active user**. Run these queries in the SQL Editor every week.

Weekly health check

```
-- Recipient confirmation rate, last 30 days
select round(100.0 * count(*) filter (where status <> 'self_reported')
             / nullif(count(*), 0), 1) as confirmation_rate_pct
from posts where created_at > now() - interval '30 days';

-- Verified positive outcomes per active user, last 30 days
select (select count(*) from reputation_events
        where kind = 'deed_confirmed' and created_at > now() - interval '30 days')::float
     / nullif((select count(distinct user_id) from reputation_events
        where created_at > now() - interval '30 days'), 0) as outcomes_per_active_user;

-- Repeat helpers: people who posted at least two deeds
select count(*) as repeat_helpers from (
  select user_id from reputation_events
  where kind = 'deed_posted' group by user_id having count(*) >= 2) t;

-- Reports per 1,000 posts
select round(1000.0 * (select count(*) from reports where target_type = 'post')
             / nullif((select count(*) from posts), 0), 1) as reports_per_1000_posts;

-- Help requests completed, and how long they took
select count(*) as completed from help_requests where status = 'completed';
```

Metric	What it tells you
Monthly and weekly active users	Community size and engagement
Good deeds submitted	Participation
Recipient confirmation rate	Trust
Help requests completed	Real-world usefulness
Repeat helpers	Retention. If people post once for popularity and leave, the mission is not working.
Reports per 1,000 posts	Safety
Time to fulfill help requests	Community responsiveness

Watch for problems during the pilot: do recipients feel comfortable confirming, are posts staged for attention, are requests being exploited. Fix these before public launch.

20. After version 1

Feature	How to approach it
Kindness chain	A chain_links table connecting two posts. Both participants must confirm the link before it shows, and the app never claims cause unless confirmed.
Daily challenges	A challenges table with one prompt per day, plus a deed form pre-filled from the prompt. Do not tie rewards to likes.
Advanced moderation	Add automated checks for spam and unsafe images that send items to the human queue. Humans keep the final decision.
Organization dashboards	A web dashboard for NGOs with campaigns and verified outcome reports.
Sponsored campaigns	Clearly labelled. Sponsors can never buy reputation for individual users.
Donations	Only after legal advice on payment and fundraising rules.

21. A 16-week plan

This is an indicative timeline for a solo beginner working steadily. Expect it to take longer, especially for moderation, iOS testing and launch preparation.

Weeks	Focus	You are done when
1-2	Research, interviews, PRD, name	You have 20 interviews and a written MVP scope
3-4	Figma wireframes and prototype	Someone completes the help-to-confirmation journey on the prototype
5	Tool setup (Chapters 3 to 5)	The starter app runs on your phone and Supabase is connected
6	Database and security rules (Chapters 6 and 7)	All tables exist with RLS on
7-8	Sign up, feed, create deed, photos, likes, comments (Chapters 8 to 11)	Two accounts can post and see each other's deeds
9-10	Help requests, confirmation, badges, leaderboard (Chapters 12 to 14)	The full journey works end to end
11-12	Report, block, moderation, privacy, security tests (Chapters 15 to 17)	Every row of the test table passes
13-16	Store builds and the pilot (Chapters 18 and 19)	Pilot users are active and you reviewed the first metrics

Appendix A. When something breaks

What you see	Likely cause	What to do
"new row violates row-level security policy"	A policy check failed, or you are not signed in	Read the policy's with check clause, and confirm the user id matches author_id
A list is empty though rows exist	No select policy allows them, or you queried posts instead of feed_posts	Read from feed_posts, or add the correct select policy
"Network request failed" on the phone	Wrong URL or missing env values	Check .env, names start with EXPO_PUBLIC_, restart with npx expo start -c
Sign-up works but there is no session	Email confirmation is on	Confirm the email, or turn it off for testing only
Image upload fails	The storage path does not start with your user id	Use userId/filename.jpg and check the storage policy
Edge Function returns 401	JWT verification is on for a public function	Deploy with --no-verify-jwt for confirm-deed only
Store rejects the app	Missing report, block, deletion, or privacy policy	Add them, then reply to the reviewer with where to find each

What you see	Likely cause	What to do
Reputation does not change	The trigger is missing or the status never changed	Check the deed_events trigger and that confirm_deed returned true

Appendix B. Glossary

Term	Plain meaning
MVP	The smallest version that proves the idea.
Expo / React Native	Tools to build Android and iOS apps from one JavaScript codebase.
Supabase	A hosted backend with a database, sign-in and file storage.
RLS (Row Level Security)	Rules inside the database that decide who can see or change each row.
Anon key / service_role key	The public key that goes in the app / the secret key that must never leave the server.
Trigger	Database code that runs automatically when a row is added or changed.
View	A saved query that looks like a table. Used here as the safe way to read posts.
Edge Function	A small piece of server code that can use secret keys safely.
EAS	Expo's cloud service that builds and updates your app.

Next step: turn the blueprint into the Product Requirements Document (Chapter 2.2), then follow Chapters 3 to 5 to get a running app on your phone before building anything else.